

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Evaluating the Security of AMD SEV-SNP Against Cache-Directed DMA and MMIO Rerouting Attacks

Alessandro Vezzù

Semester Thesis – August 4, 2026.
Institute of Information Security,
Secure & Trustworthy Systems Group.

Supervisors: Prof. Dr. Shweta Shinde
M.Sc. Benedict Schlüter



Abstract

Direct Memory Access enables high performance data transfers by allowing peripheral devices to access system memory without the involvement of the CPU. However, modern DMA implementations introduce complex security challenges. A malicious or compromised device can read or modify arbitrary regions of memory, bypassing operating system protections. To enhance system security, AMD introduced Secure Encrypted Virtualization with Secure Nested Paging, a hardware based technology that provides confidentiality and integrity protection for virtual machines through per virtual machine encryption keys and the use of Reverse Map Tables. Despite these enhancements, recent research has demonstrated that vulnerabilities may still be exploitable during SEV-SNP initialization, allowing a malicious hypervisor to subvert protection mechanisms. At the same time, new I/O technologies such as Smart Data Cache Injection change how DMA interacts with system memory by allowing data to be written directly into the processor cache rather than into main memory, which may reintroduce potential attack surfaces within protected memory regions. This thesis investigates whether modern DMA mechanisms, particularly Smart Data Cache Injection, and rerouting of MMIO transactions, can compromise the protection guarantees of SEV-SNP. It presents a detailed architectural analysis of the data fabric and its interaction with cache directed DMA operations. Using a programmable peripheral device, we design and implement a prototype capable of generating raw transaction packets with processing hints and steering information to perform cache injection and routing experiments. We evaluate several techniques to influence or overwrite protected memory structures and explore register level modifications in the IOHUB and the IOMS components to reroute MMIO transactions toward the memory controller. Our experimental results show that the Trusted Memory Region enforcement and platform level protection mechanisms effectively prevent unauthorized cache injection and memory modification. Attempts to alter routing behavior or target privileged regions were consistently blocked or caused hardware fault responses. These findings confirm the robustness of SEV-SNP against emerging DMA based attack vectors and provide empirical evidence that its architectural safeguards remain resilient even in the presence of advanced cache directed I/O capabilities.

Contents

Acronyms vii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Related Work . . . 2
- 1.3 Contribution . . . 3
- 1.4 Organization of this Thesis . . . 4

2 Background 5
- 2.1 Fundamentals of PCIe and DMA . . . 5
  - 2.1.1 PCIe Basics . . . 5
  - 2.1.2 DMA Fundamentals . . . 7
- 2.2 Smart Data Cache Injection . . . 8
  - 2.2.1 How SDCI Differs from Classic DMA . . . 9
    - 2.2.1.1 DMA Transaction Flow without SDCI . . . 9
    - 2.2.1.2 DMA Transaction Flow with SDCI . . . 10
  - 2.2.2 PCIe TLP Processing Hints . . . 10
- 2.3 AMD SEV-SNP . . . 12
  - 2.3.1 High-level Goals of SEV, SEV-ES, SEV-SNP . . . 12
  - 2.3.2 Platform Security Processor . . . 13
  - 2.3.3 Memory Protection Primitives . . . 13
    - 2.3.3.1 Reverse Map Table . . . 14
    - 2.3.3.2 Trusted Memory Regions . . . 14
- 2.4 Threat Model . . . 14

3 Implementation 17
- 3.1 Software & Hardware Environment . . . 17
  - 3.1.1 Screamer PCIe Squirrel . . . 17
  - 3.1.2 BIOS and Kernel Configuration . . . 17
- 3.2 SDCI Prototype Implementation . . . 18
  - 3.2.1 Device-side Preparation . . . 18
  - 3.2.2 Constructing and Sending SDCI TLPs . . . 19
  - 3.2.3 Privileged Memory Writes Targeting the RMP . . . 20
- 3.3 MMIO Access and Routing . . . 21
  - 3.3.1 IOHUB MMIO Register Modifications . . . 21
  - 3.3.2 IOMS Routing Register Modifications . . . 23

4 Results **25**
4.1 Smart Data Cache Injection 25
4.1.1 Experimental Validation and Microbenchmarks 25
4.1.2 Overwriting Privileged Memory 27
4.2 MMIO Access and Routing 28
4.2.1 IOHUB MMIO Register 28
4.2.2 IOMS Routing Register 29

5 Conclusion **31**

List of Figures **33**

List of Tables **34**

List of Listings **35**

Bibliography **36**

Acronyms

ATS Address Translation Services.

BAR Base Address Register.

CCM Core Coherent Master.

CCX Core Complex.

CS Coherent Slave.

CVM Confidential Virtual Machine.

DMA Direct Memory Access.

GPA Guest Physical Address.

HPA Host Physical Address.

IOMMU I/O Memory Management Unit.

IOMS IO Master Slave.

IOTLB I/O Translation Lookaside Buffer.

IOVA I/O Virtual Address.

MMIO Memory-Mapped I/O.

MWr Memory Write.

NIC Network Interface Card.

PCIe PCI Express.

PCIe ID PCIe Identifier.

PIE Power Management, Interrupts, Etc..

PSP Platform Security Processor.

RMP Reverse Map Table.

SDCI Smart Data Cache Injection.

SEV-ES Secure Encrypted Virtualization with Encrypted State.

SEV-SNP Secure Encrypted Virtualization with Secure Nested Paging.

TCB Trusted Computing Base.

TLP Transaction Layer Packet.

TPH Transaction Layer Packet Processing Hints.

UMC Unified Memory Controller.

VM Virtual Machine.

1 Introduction

This chapter motivates our study by analyzing how new DMA capabilities, including AMD's Smart Data Cache Injection (SDCI), intersect with SEV-SNP's protection model. We discuss prior attacks on DMA and SEV-SNP, setting the stage for our investigation into potential integrity risks introduced by modern I/O mechanisms.

1.1 Motivation

Direct Memory Access (DMA) allows peripheral devices to read from and write to main memory without CPU intervention. While this feature enables high-performance data transfers, it also introduces multiple attack vectors. A malicious or compromised device can exploit DMA to access or modify arbitrary memory regions, thereby bypassing operating system–level protections.

Prior research has shown that even when the I/O Memory Management Unit (IOMMU) functions correctly, malicious peripherals can still leverage DMA to manipulate kernel pointers, exploit shared allocators, and abuse features such as Address Translation Services (ATS). These attacks may lead to data leakage, arbitrary code execution, or privilege escalation within the kernel [MRG+19].

To improve security and isolation in virtualized environments, AMD introduced Secure Encrypted Virtualization with Secure Nested Paging (SEV-SNP). This hardware-based technology provides confidentiality and integrity protection for virtual machines. Confidentiality is achieved through unique per-VM encryption keys, while integrity is enforced using the Reverse Map Table (RMP), which records ownership and access permissions for all DRAM pages in the system.

Since the RMP resides in DRAM, it must be protected against tampering during system initialization. AMD mitigates this risk by delegating RMP setup to a trusted coprocessor called the Platform Security Processor (PSP), which establishes temporary protection regions known as Trusted Memory Regions (TMRs) until the RMP itself becomes capable of enforcing access control [Adv20].

Despite these protections, studies such as Heracles [SWS25] and WeSee [SSBS24] demonstrate that SEV-SNP can still be compromised by a malicious hypervisor.

More recently, RMPocalypse [SS25] exposed weaknesses in the platform's initialization phase, revealing that the RMP is briefly vulnerable before full enforcement begins. During this period, an adversary with privileged host access can alter RMP entries to corrupt its structure. Even a single modified entry can invalidate SEV-SNP's integrity guarantees, allowing attackers to forge attestation data, enable debugging functionality, or read and modify encrypted guest memory arbitrarily.

Meanwhile, new I/O acceleration technologies continue to change how devices interact with CPU caches and system memory. AMD's SDCI allows incoming I/O data to be written directly into CPU caches (L2 or L3) instead of DRAM. This process relies on PCI Express (PCIe) Transaction Layer Packet Processing Hints (TPH) and steering tags to control cache placement [Adv25b]. Although SDCI is intended to improve latency and throughput, it introduces a new channel through which DMA operations can affect cache state and potentially influence protected memory contents.

Combined with complex platform routing logic that governs Memory-Mapped I/O (MMIO) and memory transactions, these new capabilities may allow malicious devices to reroute or inject DMA traffic into privileged cache or DRAM regions.

These observations motivate our investigation: Can a malicious PCIe device, leveraging modern DMA features such as SDCI and MMIO rerouting, modify or corrupt SEV-SNP's protected structures during initialization or runtime? Answering this question provides critical insight into the resilience of SEV-SNP's hardware protections against emerging DMA technologies and helps identify whether performance-driven I/O innovations inadvertently reintroduce previously mitigated attack surfaces.

1.2 Related Work

Several previous studies have examined weaknesses in AMD SEV-SNP's memory protection and I/O isolation mechanisms, providing important context for this work.

Thunderclap [MRG+19] investigated DMA-based privilege escalation attacks on general-purpose systems and showed that PCIe's Address Translation Service (ATS) can be abused to bypass the IOMMU, granting unrestricted access to system memory. This work demonstrated that *features designed for performance can be exploited for privilege escalation* when not properly isolated. Although Thunderclap does *not* target AMD SEV-SNP, it illustrates how advanced I/O features introduced for performance can unintentionally reintroduce attack surfaces. This highlights the importance of evaluating mechanisms like AMD's Smart Data Cache Injection, which also aims to improve performance, for comparable security risks.

RMPocalypse [SS25] revealed that AMD SEV-SNP can be compromised during system initialization due to incomplete enforcement of memory-access barriers. Before the Reverse Map Table enforcement becomes fully active, RMP memory remains writable by x86 cores, allowing *dirty cachelines created during initialization* to be flushed afterward and overwrite the initial RMP state. This finding shows that even brief gaps in hardware protection during system startup can compromise the overall security of the platform. Our work builds on this insight by exploring whether AMD's Smart Data Cache Injection (SDCI), which also interacts with CPU caches, could create similar dirty cachelines during initialization and thus expose comparable integrity risks.

BadDRAM [DMWO+25] uncovered a weakness in AMD SEV-SNP by manipulating the DRAM's SPD configuration to create false memory mappings. By *making the DRAM appear larger than it is*, the attack induces *hardware aliasing*, where multiple physical addresses map to the same DRAM cells, breaking SEV-SNP's integrity guarantees. The work demonstrates that SEV-SNP's security fundamentally depends on correctly configured and isolated memory hardware, motivating our investigation into whether modifying I/O configuration registers could trigger comparable aliasing that compromises memory integrity.

1.3 Contribution

This work makes the following key contributions:

- We provide a comprehensive architectural analysis of the PCIe and data fabric path involved in SDCI, identifying the critical hardware components, that influence cache-directed DMA operations.
- We design and implement a practical SDCI prototype using a programmable PCIe endpoint (Screamer PCIe Squirrel) and the Linux `CONFIG_PCIE_TPH` subsystem.
- We propose and experimentally evaluate several new approaches to bypass or influence AMD SEV-SNP memory protection primitives (RMP and TMR) using SDCI-based cache injection.
- We develop and perform MMIO routing experiments by modifying IOHUB and IO Master Slave (IOMS) registers to investigate whether MMIO transactions can be redirected toward the memory controller.
- We present experimental results that validate correct SDCI operation, quantify its latency characteristics, and empirically demonstrate that current SEV-SNP mechanisms effectively prevent unauthorized memory modification, while malformed or misrouted packets do result in system failures.

1.4 Organization of this Thesis

Chapter 2 introduces the architectural foundations necessary for understanding the mechanisms analyzed in this study. It explains the fundamentals of PCIe communication and DMA, then details AMD's SDCI feature and the SEV-SNP protection framework, concluding with the threat model adopted for this work.

Chapter 3 describes the experimental setup and implementation, including the hardware and software environment, SDCI prototype design, and methods used to construct and transmit Transaction Layer Packets (TLPs) with processing hints and steering tags. It also outlines our techniques for targeting privileged memory and rerouting MMIO traffic toward the memory controller.

Chapter 4 presents and analyzes the experimental results. We evaluate the performance and correctness of SDCI, examine its potential for bypassing SEV-SNP protections, and discuss the outcomes of our MMIO rerouting experiments.

Finally, Chapter 5 summarizes the key findings of this work. It concludes that AMD SEV-SNP effectively prevents unauthorized cache injection and MMIO redirection, highlighting the robustness of the platform's TMR enforcement and hardware-level protection mechanisms.

2 Background

This section provides the necessary architectural and conceptual foundations for understanding the mechanisms explored in this work. It begins by outlining the fundamentals of PCIe communication and DMA, which describe how peripheral devices exchange data with system memory. Building on this, it introduces AMD's Smart Data Cache Injection technology, explaining how it enhances traditional DMA by enabling direct cache-level data placement. Finally, the section summarizes key aspects of AMD SEV-SNP, the platform's hardware-based memory protection framework, and defines the threat model used throughout this study.

2.1 Fundamentals of PCIe and DMA

2.1.1 PCIe Basics

PCI Express (PCIe) is a high-speed serial interconnect standard used to connect peripherals such as Network Interface Cards (NICs), GPUs, and NVMe storage controllers to a host system. The fundamental unit of PCIe communication is the link, which interconnects components of different types. Conceptually, PCIe follows a layered and hierarchical architecture, as illustrated in Figure 2.1.

At the top of this hierarchy resides the Root Complex, a host-side controller, typically integrated into the CPU or chipset, that bridges the processor and memory subsystem to the PCIe fabric.

The Root Complex itself exposes multiple ports, each representing a separate hierarchical domain. Each domain can consist of a single endpoint device or multiple endpoints interconnected via PCIe switches. Host software (for example, the operating system or a hypervisor) configures these devices through their configuration space, assigning Base Address Registers (BARs) as described in Section 2.1.1 [Rot17].

Transaction Layer Packet (TLP) Communication over PCIe occurs through Transaction Layer Packets (TLPs). Two primary TLP types exist, determined by their target address space:

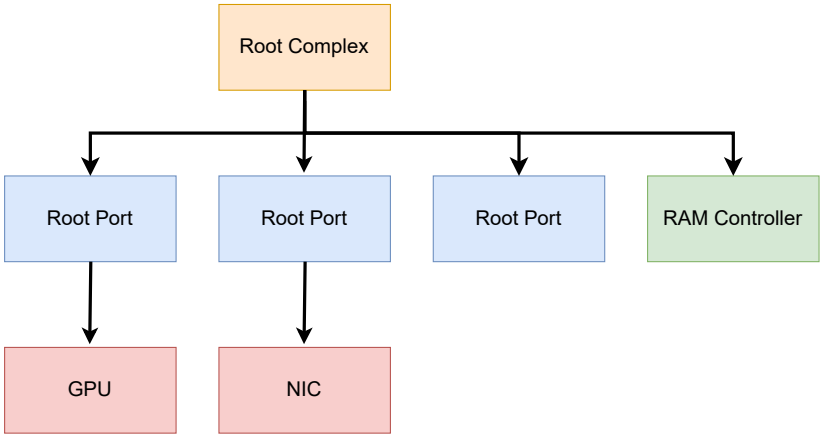


Figure 2.1: Simplified architecture of a PCIe Root Complex and its connection hierarchy [Ota23].

- **Memory Read / Memory Write TLPs:** Used to send read and write requests to the system's memory subsystem.
- **Configuration and I/O TLPs:** Used for device enumeration, initialization, and configuration.

Some requests, such as memory reads, may generate a completion response, either carrying data or serving as an acknowledgment. If a request is sent to an endpoint incapable of handling it, a special completion packet is returned, known as an *unsupported request* response [PS19].

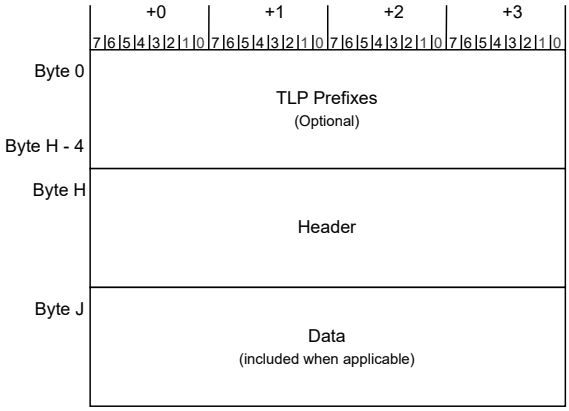


Figure 2.2: Structure of a PCIe TLP.

As illustrated in Figure 2.2, a TLP consists of a header and an optional data pay-

load. Additionally, a TLP may also include an optional prefix. Not all TLPs carry data, for instance, a Memory Read request only contains the header specifying the target address, while the data is returned later in the completion [Rot17, PS19].

To ensure that completions are delivered to the correct device, each TLP header includes a unique PCIe Identifier (PCIe ID). The PCIe ID consists of an 8-bit bus number, a 5-bit device number, and a 3-bit function number, as shown in Figure 2.3. Switches in the PCIe fabric use this identifier to route packets to the correct endpoint.

Base Address Registers (BARs) PCIe devices expose various capabilities, such as Address Translation Services (ATS) or Transaction Processing Hints (TPH), within their configuration space [PS19]. They also advertise one or more Base Address Registers, which reserve specific regions of the system's physical address space. During enumeration, the operating system assigns these regions and maps them as MMIO areas. Software communicates with device registers and on-device memory through these BAR-mapped addresses.

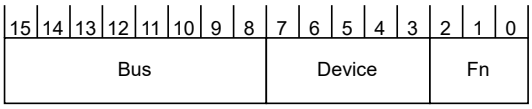


Figure 2.3: Structure of a PCIe Identifier (Bus, Device, and Function numbers).

2.1.2 DMA Fundamentals

Direct Memory Access (DMA) enables PCIe peripheral devices to perform memory reads and writes independently of the CPU, allowing high-throughput data transfers without direct processor involvement. Device drivers configure DMA-capable devices with buffer addresses and control descriptors. Once configured, the device issues memory read or write TLPs to transfer data between its local buffers and host memory [MRG+19]. Upon completion, devices typically trigger interrupts to signal transfer completion [PS19].

IOMMU The I/O Memory Management Unit (IOMMU) resides between PCIe devices and the system memory controller and serves three key functions:

- **Address translation:** Maps I/O Virtual Addresses (IOVAs) to physical DRAM addresses, allowing the operating system to control device memory access.

- **Isolation:** Prevents faulty or malicious devices from issuing unauthorized DMA operations.
- **Remapping and virtualization:** Enables safe device sharing across multiple Virtual Machines (VMs) through independent translation tables.

There are several major implementations of IOMMUs, including Intel VT-d [Int14], AMD-Vi [Adv15], and Arm System MMU [ARM16]. Despite architectural differences, all provide similar functionality. They all virtualize device address spaces and translate IOVAs into physical addresses, caching recent translations in the I/O Translation Lookaside Buffer (IOTLB) for performance optimization. If a requested address is not mapped within the IOMMU, the access results in a page fault. Consequently, the IOMMU provides a critical layer of protection against faulty or malicious DMA transactions [MRG+19].

Coherent vs. Non-Coherent Transfers A DMA transfer may interact with CPU caches. Depending on the platform and configuration, DMA operations can be either cache-coherent or non-coherent.

In a coherent DMA transfer, hardware ensures cache coherence between PCIe devices and the CPU. This means that DMA reads and writes are properly synchronized with cached data, and cache lines may be flushed or invalidated automatically when required.

In contrast, a non-coherent DMA transfer does not maintain cache consistency automatically. In this case, software must explicitly flush or invalidate the CPU caches before and after DMA operations to ensure both the CPU and the device access up-to-date data.

PCIe also defines snoop hints to indicate whether a given transaction should participate in cache coherence [PS19].

2.2 Smart Data Cache Injection

Eliminating the CPU from the packet processing path through DMA enables data transmission rates of tens of gigabytes per second at sub-microsecond latencies in modern NICs [HDGT+17]. Nevertheless, fetching data from system memory remains extremely expensive in terms of latency and bandwidth. Therefore, to achieve faster application processing, it is essential to optimize packet handling by minimizing memory access latency and DRAM bandwidth consumption.

One effective strategy to alleviate pressure on DRAM is to reduce the number of memory accesses by keeping frequently used data closer to the processor. Modern systems achieve this through multi-level cache hierarchies that serve as high-speed intermediaries between the CPU and main memory.

The effectiveness of a cache depends on storing data that is both frequently accessed and highly useful. However, determining which data should be cached is often challenging, especially when host hardware and processing resources have limited visibility into I/O operations.

To address this challenge, AMD's latest Zen 5 processors introduce **Smart Data Cache Injection (SDCI)**, a mechanism that allows I/O data to be preloaded directly into a CPU core's L2 or L3 cache. This proactive caching reduces access latency and improves performance for latency-sensitive workloads [Adv25b].

2.2.1 How SDCI Differs from Classic DMA

To illustrate how SDCI differs from a traditional DMA request, we will examine the transaction flow of each method and highlight their key components. As shown in Figure 2.4, nearly all system components are interconnected through the **Data Fabric**, which connects x86 cores, memory controllers, and I/O subsystems.

2.2.1.1 DMA Transaction Flow without SDCI

Without Smart Data Cache Injection, the DMA transaction flow proceeds as illustrated in Figure 2.4. The DMA request originates from a PCIe-attached device such as a NIC, SmartNIC, or FPGA board. The device issues a TLP request, which is routed through multiple system components, starting with the address translation in the IOMMU.

After address translation, the request passes through the **IOHUB** and the **I/O Master-Slave** (IOMS). The IOHUB resides below the data fabric and is responsible for routing requests between the IOMMU and the system interconnect. The IOMS directly connects the data fabric with the IOHUB and ensures correct routing of MMIO requests within the data fabric.

From the IOMS, the packet travels across the bus to the **Coherent Slave (CS)**. The CS is responsible for maintaining cache coherence across all CPU cores. Incoming requests to the CS block are first enqueued and processed sequentially. The CS checks whether the requested data is currently cached in any core by consulting a snoop table.

In the case of a cache coherent DMA request, if the data already resides in a cache, the CS invalidates the corresponding cache line in the affected core to maintain coherence. Finally, from the CS block, the DMA request reaches the **Unified Memory Controller (UMC)**, which is responsible for performing actual read and write operations to and from physical DRAM [SS25].

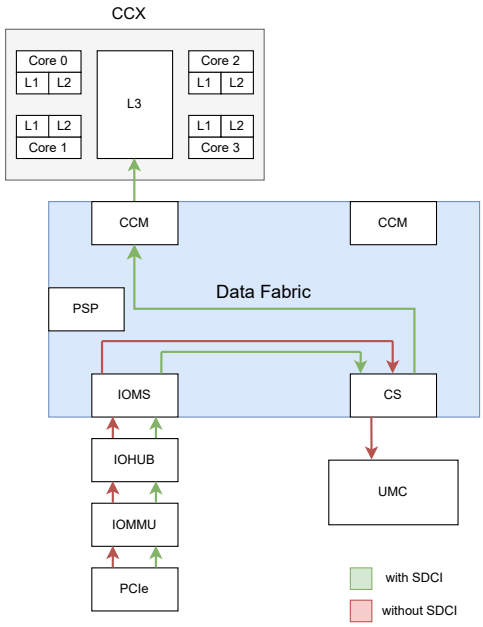


Figure 2.4: Simplified DMA transaction flow within the data fabric, with and without Smart Data Cache Injection.

2.2.1.2 DMA Transaction Flow with SDCI

When SDCI is enabled, the DMA transaction flow remains identical up to the CS block. However, instead of forwarding the request to the memory controller, the CS block routes it directly to the **Core Coherent Master (CCM)**. The CCM connects the Core Complex (CCX) to the data fabric and manages memory routing on behalf of the CPU cores. From the CCM, the incoming data is directly injected into the appropriate cache (L2 or L3) [SS25]. Eventually, when the cache is flushed or evicted, the data is written back to main memory. This approach allows data to be immediately available to the CPU, significantly reducing access latency for time-sensitive workloads [Adv25b].

2.2.2 PCIe TLP Processing Hints

When SDCI is active, the CS block must determine which data should be injected and into which CPU cache it should be placed. To make this decision, SDCI utilizes the PCIe TLP Processing Hints (TPH) feature, an open industry standard that defines how a device can provide caching and locality hints to the host.

TPH introduces a set of bits within the TLP header that specify how the associated

data should be handled. As summarized in Table 2.5, four distinct processing hint types are defined.

Table 2.5: Description of PCIe TLP Processing Hint Types

Value	Description
00	Bi-directional data structure. Indicates frequent read and/or write access to data by both the host and the device.
01	Requester. Indicates frequent read and/or write access primarily by the device.
10	Target. Indicates frequent read and/or write access primarily by the host.
11	Target with Priority. Indicates frequent host access with high temporal locality for the associated data.

In addition to the processing hint, the CS block also needs information about the target cache. This information is conveyed through a steering tag, which encodes the destination core and cache level. Using this steering tag, the CS block determines exactly which cache should receive the injected data.

Figure 2.6 illustrates an example of a TLP packet with TPH information. The processing hints (denoted as PH) are appended to the address field. To signal the presence of these PH bits, the **TH** bit in the TLP header must be set to one [PS19].

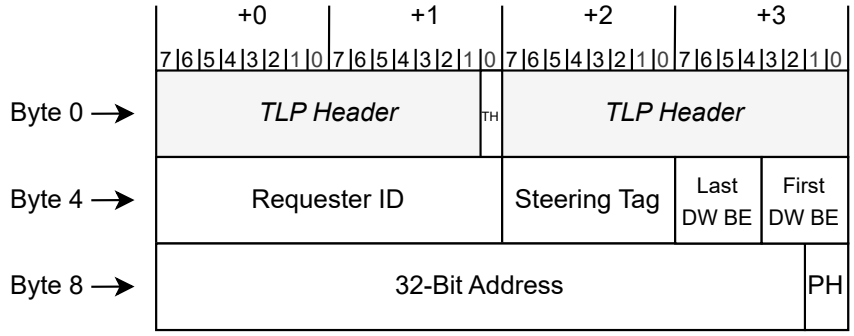


Figure 2.6: Example structure of a TLP containing TLP Processing Hints and steering information.

2.3 AMD SEV-SNP

AMD Secure Encrypted Virtualization with Secure Nested Paging (SEV-SNP) is an advanced hardware-based security technology designed to protect virtual machines from unauthorized access, even from privileged software such as the hypervisor. These protected virtual machines are referred as Confidential Virtual Machines (CVMs). The primary feature of SEV-SNP is its ability to provide confidentiality and integrity guarantees for VM memory [Adv20].

At a high level, the platform relies on a trusted processor component, the AMD Platform Security Processor (PSP), to provision encryption keys, verify integrity, and manage memory metadata through specialized data structures. Two key primitives central to our work are the Reverse Map Table (RMP) and Trusted Memory Regions (TMRs).

The RMP stores ownership and mapping information for each physical page in memory. It ensures that a confidential virtual machine can only access pages explicitly assigned to it, preventing even a privileged hypervisor from remapping or inspecting encrypted pages.

TMRs, by contrast, define trusted physical memory regions and enforce access control by filtering memory requests based on their origin, preventing untrusted components from accessing or modifying secure regions [SS25].

A deep understanding of how RMP state transitions occur, and how cached memory states may influence integrity verification, is essential for evaluating whether microarchitectural mechanisms such as SDCI could inadvertently violate SEV-SNP's intended protection boundaries.

2.3.1 High-level Goals of SEV, SEV-ES, SEV-SNP

The SEV framework has evolved through several generations, each iteration addressing distinct limitations and expanding the protection scope.

The initial AMD SEV design enables confidential virtual machines through memory encryption, assigning each VM a unique AES key. Any attempt by the hypervisor to read a CVM's memory would only reveal its encrypted contents [Adv20, PDH25].

However, the CPU register state remained unencrypted, allowing the hypervisor to inspect or modify data used by the VM. To address this limitation, Secure Encrypted Virtualization with Encrypted State (SEV-ES) was introduced, providing additional protection by encrypting the CPU register state as well [Kap17, HB17].

While SEV-ES improved confidentiality, it did not provide strong integrity guarantees. SEV-SNP addressed this limitation by adding hardware-level integrity protection, preventing hypervisors from performing memory remapping, replaying old data, or aliasing multiple guest physical addresses to a single system physical page. These protections are enforced through the Reverse Map Table (RMP), which validates every access to protected memory [MPR+21, MHHW18, Adv20].

2.3.2 Platform Security Processor

The AMD Platform Security Processor (PSP) is an embedded co-processor that functions as the hardware root of trust for AMD systems. It is the most privileged entity in the platform and operates independently of the main CPU cores. Its services can be accessed by the hypervisor through MMIO registers at runtime.

Running signed firmware supplied by AMD, the PSP is responsible for establishing the hardware root of trust by performing early integrity checks, initializing the memory encryption engines, generating the SNP key, and setting up memory protection primitives such as the Reverse Map Table and Trusted Memory Regions (TMRs) during system boot [SS25].

2.3.3 Memory Protection Primitives

As mentioned above, our work is particularly interested in the Reverse Map Tables and Trusted Memory Regions. Therefore, we specifically focus on these memory protection primitives.

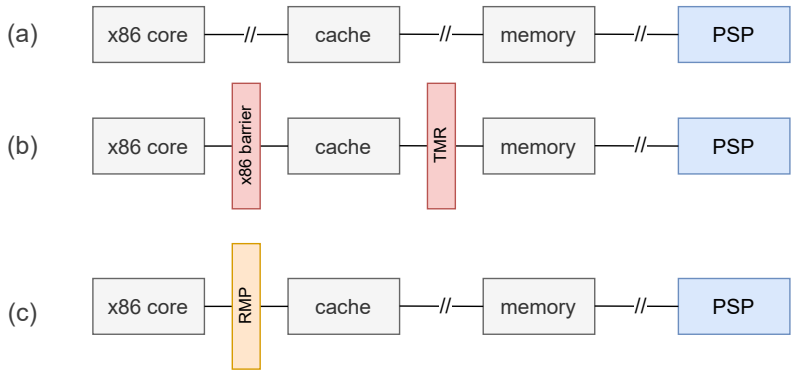


Figure 2.7: (a) RMP memory is writable before SNP initialization. (b) The PSP sets barriers blocking writes from x86 cores and devices. (c) After RMP initialization, x86 cores uses RMP semantics [SS25].

2.3.3.1 Reverse Map Table

The RMP maintains a secure mapping between host physical addresses (HPAs) and guest physical addresses (GPAs) and ensure that any memory access by a confidential virtual machine is verified against the intended mapping. If a CVM attempts to access a memory region for which it does not have permission, an exception is raised. This mechanism also prevents the hypervisor from remapping or swapping a CVM's memory pages.

The RMP is initialized and managed by the Platform Security Processor, which is the only component authorized to modify it. As illustrated in Figure 2.7, during initialization, the PSP establishes memory barriers that block all read and write access from the x86 cores. Additionally, a second barrier is configured using TMRs, see Section 2.3.3.2, to prevent both x86 cores and peripheral devices (for example, a DMA-capable device) from modifying the RMP. Once the RMP is fully configured, these barriers are lifted, and RMP checks become active.

To protect the integrity of its own structure, the RMP includes a self-referential entry that prevents unauthorized modification. Any legitimate update to the RMP is performed through the `RMPUPDATE` instruction. Hardware-level access control ensures that this instruction cannot be abused by unprivileged software. If a hypervisor lacks the necessary permissions, the PSP performs the update on its behalf [SS25].

2.3.3.2 Trusted Memory Regions

Trusted Memory Regions (TMRs) provide an additional layer of protection by filtering all memory requests based on their origin. Each transaction within the data fabric carries source identifiers that allow the TMR logic to determine whether a request should be permitted or blocked.

Only the PSP is authorized to configure or modify TMR entries. During the system boot, the PSP uses TMRs to protect sensitive structures such as the RMP from both CPU and peripheral writes.

TMRs do not distinguish between x86 and microcode access [Adv23]. This is important for our work, as if microcode can write to a specific memory region, the hypervisor might also be able to perform such writes, for example through a cache flush [SS25].

2.4 Threat Model

Our threat model assumes a local adversary with full physical access to the target platform. The attacker can connect an arbitrary, malicious PCIe peripheral

and exercise complete control over its functionality. Specifically, the adversary can:

- Craft and inject arbitrary Transaction Layer Packets (TLPs)
- Perform DMA transactions with full control over addresses, payloads, and steering information
- Enable or manipulate PCIe features such as SDCI and TPH to influence cache placement
- Modify the host kernel, load custom drivers, and alter IOMMU mappings to facilitate attacks

Despite this elevated privilege, the Trusted Computing Base (TCB) remains intact. The AMD PSP, SEV-SNP attestation authority, and firmware responsible for root-of-trust establishment are assumed uncompromised. We explicitly exclude the following from our threat scope:

- Physical tampering with DRAM modules or CPU packages
- Modification of CPU microcode or PSP firmware
- Cache side-channel attacks using SDCI

3 Implementation

This section describes the hardware and software testbed and the experiments we performed. We first introduce the Screamer PCIe Squirrel endpoint and the tooling used to craft raw TLPs (Google's `go-pcie-tlp`), then detail BIOS and Linux kernel configuration required for SDCI experiments. Subsequent sections present the SDCI prototype implementation, covering device-side preparation, TLP construction, and experimental procedures for targeting the Reverse Map Table (RMP) and exploring MMIO rerouting through the I/O subsystem.

3.1 Software & Hardware Environment

3.1.1 Screamer PCIe Squirrel

All experiments were conducted using a Screamer PCIe Squirrel endpoint device [Lam25]. The Screamer PCIe Squirrel is an FPGA-based PCIe board that can be programmed with custom firmware. The key feature used in this work is its ability to craft and issue raw PCIe TLPs directly from the device. For this purpose, we used Google's `go-pcie-tlp` library [Goo23], which provides a convenient interface for constructing and transmitting TLPs.

3.1.2 BIOS and Kernel Configuration

BIOS Settings The platform BIOS was configured according to the recommendations provided in AMD's *Smart Data Cache Injection (SDCI) White Paper* [Adv25b]. The appendix of the white paper describes the required system configuration and BIOS options for setting up an environment that supports SDCI experiments. Only the BIOS settings explicitly mentioned in the appendix were applied, no additional modifications were made.

Linux Kernel Support for PCIe Transaction Processing Hints (TPH) was upstreamed into the Linux kernel starting with version 6.13. For our experiments, we built and used the Linux kernel `6.15.0-rc5` [Lin23]. When compiling the kernel, the following configuration options must be enabled:

- `CONFIG_PCIE_TPH`: Enables support for PCIe Transaction Processing Hints [vTHtLkc24].
- `CONFIG_X86_CPU_RESCTRL`: Enables CPU resource control features such as AMD Platform QoS, allowing adjustment of prioritization between L2 and L3 cache injection [YLStLkc16].

Enabling `CONFIG_PCIE_TPH` is essential, as it provides the kernel interfaces required to query and use TPH steering tags for cache-directed memory accesses. For older Linux versions, it is also possible to apply the necessary kernel patch to add TPH support.

In addition, to obtain more detailed information about system failures, we applied the patch `x86/CPU/AMD: Print the reason for the last reset` [Lar25], which extends kernel diagnostics to include the cause of the most recent system reset. This additional information was useful for post-crash analysis during development and testing.

3.2 SDCI Prototype Implementation

3.2.1 Device-side Preparation

Not all PCIe endpoints natively support or expose the Transaction Processing Hints (TPH) capability. The Screamer PCIe Squirrel, by default, lacks TPH advertisement in its configuration space. Nevertheless, an endpoint remains suitable for SDCI experimentation if the following two criteria are met:

1. The device must be able to insert an appropriate steering tag and TPH into outgoing TLPs.
2. The host platform must provide steering information that associates a TLP with a specific CPU or cache domain.

The first requirement is fulfilled, as illustrated in Listing 3.1, which demonstrates that raw TLP packets can be constructed and transmitted over PCIe using the Screamer PCIe Squirrel.

The second requirement is handled through the Linux PCIe subsystem. The steering tag mechanism is platform-specific and is therefore not standardized in the PCIe specification. However, when the Linux kernel is compiled with `CONFIG_PCIE_TPH`, it exposes a helper function that retrieves a steering tag for a given CPU:

```
int pcie_tph_get_cpu_st(struct pci_dev *pdev,
                        enum tph_mem_type type,
                        unsigned int cpu_uid,
                        u16 *tag);
```

This function fails if TPH is not properly enabled for the corresponding PCIe device through the Linux PCIe subsystem. Since the Screamer PCIe Squirrel allows manual construction of TLP packets, we bypassed this limitation by directly setting the relevant fields in the device's `struct pci_dev`, specifically `tph_req_type` and `tph_enabled`, enabling the kernel to return valid steering tags [vTHtLkc24].

3.2.2 Constructing and Sending SDCI TLPs

As discussed in Section 2.2.2, three key components of the TLP header must be properly configured to enable SDCI: the Processing Hint (PH) bits, the TPH Enable (TH) bit, and the steering tag.

To perform SDCI, we constructed PCIe Memory Write (MWr) transactions and configured each TLP as follows:

- **PH bits:** The two least significant bits of the I/O address were set to `b'10`, indicating frequent host read and/or write access (see Table 2.5).
- **TH bit:** This bit was set to `1` to indicate the presence of a processing hint in the TLP.
- **Steering tag:** The 8-bit steering tag was obtained and set using the `pcie_tph_get_cpu_st()` helper function for the target CPU core. Although this function returns a 16-bit value, which would require a TLP prefix, we used only the lower 8 bits because the upper bits were always zero.

Listing 3.1 shows a simplified Go implementation using Google's `go-pcie-tlp` library to construct and transmit a Memory Write TLP that includes the SDCI processing hint.

```
1  /*
2  Construct and send an MWr TLP carrying the SDCI hint.
3
4  Parameters:
5    - reqID: Device identifier for the PCIe endpoint
6    - rw:   Interface to the device's TLP transmit controller
7    - tag:  Steering tag obtained from the kernel
8    - addr: I/O virtual target address
9    - data: Payload data to write
10 */
11 func performSDCI(reqID genTLP.DeviceID, rw screamer.TLPController,
12                  tag uint16, addr uint64, data []byte) error {
13    // Create a new Memory Write TLP (returns a TLP object)
```

```
    tlp, err := genTLP.NewMWr(reqID, addr, data)
    if err != nil {
        return err
    }

    // Set PH in the address: clear lower two bits, then OR with 0x2 (PH = 2)
    tlp.Address = genTLP.Address((addr &^ 0x3) | 0x2)

    // Enable TPH steering hint and assign the steering tag
    tlp.TH = true
    tlp.Tag = uint8(tag & 0xff) // Truncate to 8 bits

    // Serialize and transmit the TLP to the device
    rw.Write(tlp.ToBytes())
    return nil
}
```

Listing 3.1: Example Go implementation for constructing and transmitting a Memory Write (MWr) TLP with an SDCI processing hint.

3.2.3 Privileged Memory Writes Targeting the RMP

Our first goal is to overwrite the RMP to compromise AMD SEV-SNP by injecting data into the CPU cache. After platform initialization the RMP protects itself from ordinary x86 accesses while still remaining accessible to microcode. To attempt tampering, we aimed to populate CPU caches with forged RMP entries during initialization and subsequently flush these lines after initialization to physical memory, potentially overwriting RMP contents.

Native approach As an initial experiment we used native SDCI (see Listing 3.1) without further modification. We mapped the physical base of the RMP, read from MSR via `rdmsrl(MSR_AMD64_RMP_BASE, rmp_base_address_msr)`, into the IOMMU using `iommu_map`, and then issued SDCI write requests that target the RMP.

Cache coherency Figure 2.4 shows that packets traverse the CS block, which enforces cache coherency. Because RMP pages are marked privileged, the CS block could enforce strict coherence handling, potentially preventing SDCI writes from populating CPU caches. To mitigate this, we set the *no-snoop* flag in our TLPs, which indicates that coherence actions are not required for the addressed region.

System overload The CS block buffers multiple outstanding requests and services them one after another. By saturating the system with benign SDCI writes to unprivileged memory while interleaving RMP-targeted injections, we evaluated if queued entries could still populate the cache before being processed. To further stress

the system, we also ran a kernel thread that repeatedly executes `wbinvd` (write-back and invalidate) to continuously evict caches and observe whether queued CS-block requests are flushed to memory before they are processed.

Crossing memory boundaries Because the CS block may enforce stricter checks for privileged regions, we evaluated two boundary conditions. First, we issued SDCI writes from a non-privileged region placed immediately before the protected region to test whether writes can propagate across the boundary into the privileged area. Second, we tested malformed TLP length encodings, cases where the length field in the TLP header does not match the actual payload length, to observe how the CS block and downstream logic handle writes that extend past the declared payload boundary.

Changing cachability Finally, we investigated whether the cacheability attributes of the RMP region affect injection success. The CS block may prevent cache population for regions marked non-cacheable, if so, forcing the region to be write-back cacheable could enable injection. To test this, we mapped the RMP physical address into kernel virtual memory and invoked `set_memory_wb` [kc24] to mark the pages as write-back cacheable prior to issuing injection attempts.

3.3 MMIO Access and Routing

Hardware registers are typically exposed through Memory-Mapped I/O (MMIO) regions. Device drivers map these regions into the kernel's virtual address space and interact with hardware by performing MMIO reads and writes. Access to these regions is mediated by the platform's MMIO controller, which translates memory transactions into device register operations.

MMIO regions are of particular interest because they often contain privileged memory.

Our second experimental objective was to determine whether MMIO requests could be rerouted to the system's memory controller, thereby allowing privileged MMIO regions to be treated as ordinary memory. To achieve this, we modified I/O configuration registers to redirect MMIO traffic toward the memory subsystem.

3.3.1 IOHUB MMIO Register Modifications

The IOHUB exposes the base addresses and limits that define MMIO regions through dedicated configuration registers. In this experiment we modified several IOHUB registers that determine the MMIO layout:

1. `NB_TOP_OF_DRAM_SLOT1_STRUCT`
2. `NB_LOWER_TOP_OF_DRAM2_STRUCT` and `NB_UPPER_TOP_OF_DRAM2_STRUCT`
3. `NB_TOP_OF_DRAM3_STRUCT`

The register `NB_TOP_OF_DRAM_SLOT1_STRUCT` encodes the base for the first MMIO region, the pair `NB_LOWER_TOP_OF_DRAM2_STRUCT` / `NB_UPPER_TOP_OF_DRAM2_STRUCT` and `NB_TOP_OF_DRAM3_STRUCT` encode the base and upper bound for the second MMIO region [AMD23]. A simplified memory map illustrating these regions is shown in Figure 3.2.

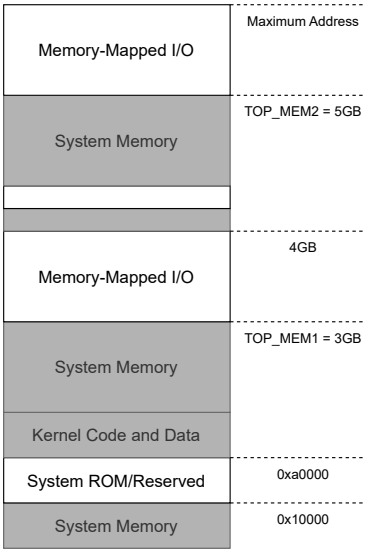


Figure 3.2: Physical address layout example [CFHB22].

Concretely, we set the first MMIO base to 4 GiB and extended the second MMIO upper bound to `0x5F0000000000`. We selected such a high upper bound to observe system behavior when addressing locations beyond actual physical memory. In particular, we sought to determine whether accesses to those high addresses are (a) forwarded to the memory controller, (b) trapped by the platform, or (c) aliased by the memory controller due to truncated address lines, as reported in prior studies [DMWO+25].

After updating the IOHUB configuration, we mapped the entire first MMIO range and portions of the (artificially) extended second MMIO range into the IOMMU and issued DMA transactions from the Screamer PCIe Squirrel. These DMA requests targeted both the legitimate MMIO region and the extended region so we could

compare observable responses and side effects produced by the memory and MMIO controllers.

3.3.2 IOMS Routing Register Modifications

As shown in Figure 2.4, PCIe transactions traverse also the I/O Master-Slave (IOMS), which routes packets based on target addresses. The IOMS maintains twenty MMIO address registers that can be used to define routing rules for MMIO requests. Each rule is specified using a combination of an MMIO Base/Limit Address Register, which encodes a particular address range, and an MMIO Address Control Register, which defines the routing behaviour for that range. If an MMIO access falls inside a defined range, the corresponding control register determines how the request is handled.

```
1  typedef union {
2    struct {
3      uint32_t RE:1;                    ///< 1: Reads to this address range are enabled.
4      uint32_t WE:1;                    ///< 1: Writes to this address range are enabled.
5      uint32_t CpuDis:1;                ///< Control whether CPU accesses get redirected to
           IO compatibility space. IO accesses ignore this field.
6      uint32_t Np:1;                    ///< Select whether CPU writes to this MMIO range go
            through posted or non-posted channel.
7      uint32_t :12;                     ///< Reserved
8      uint32_t DstFabricID:12;          ///< For transactions which hit in this range, this
           field specifies the destination FabricID of the IOS, which may be local or on a
           remote die.
9      uint32_t :4;                      ///< Reserved
10   } Field;
11   uint32_t Value;
12 } MMIO_ADDRESS_CONTROL_REGISTER;
```

Listing 3.3: MMIO Address Control Register.

The MMIO Address Control Registe contains read and write enable bits (`RE` and `WE`), as well as a `DstFabricID` field. If a request falls within the specified range and both access bits are enabled, the transaction is routed to the destination component indicated by `DstFabricID`. If either access bit is unset, the corresponding request is blocked [AMD23].

To reroute the traffic generated by the Screamer PCIe Squirrel, we modified the IOMS register of the IOMS instance responsible for this device. Specifically, we enabled read and write access by setting `RE` and `WE`, and reassigned the `DstFabricID` field for selected MMIO ranges. We redirected traffic to several targets, including the CCM, the CS block, and the Power Management, Interrupts, Etc. (PIE) block.

The PIE block is part of the data fabric and centralizes platform level power, clock and interrupt control, it is configured only by the PSP, so normal software cannot change its settings.

Finally, we combined these IOMS changes with the IOHUB register modifications described above to ensure any IOHUB specific flags did not prevent or alter the intended rerouting.

Some IOMS routing registers were unused by default, which allowed us to repurpose them to route DRAM traffic. For these registers we reprogrammed the base, limit, and `DstFabricID` fields so that requests to an allocated memory region were forwarded according to our chosen routing.

Finally, to minimize the risk of system instability caused by unrelated MMIO requests, we split a single MMIO range into multiple smaller segments using unused IOMS registers. All but one of the resulting segments were configured to reproduce the original routing behaviour, only one segment had its `DstFabricID` changed to route traffic to one of the target destinations (CCM, CS block or PIE block).

4 Results

This chapter presents the experimental results obtained from evaluating multiple approaches to accessing and modifying privileged memory and Memory-Mapped I/O (MMIO) regions. We first validate the functionality of Smart Data Cache Injection on our platform, followed by an analysis of the techniques used to attempt overwriting privileged memory structures. Finally, we discuss the effects of modifying IOHUB and IOMS routing registers.

4.1 Smart Data Cache Injection

We begin by verifying that Smart Data Cache Injection operates as expected on our system and then evaluate its impact on attempts to overwrite privileged memory regions.

4.1.1 Experimental Validation and Microbenchmarks

To confirm correct SDCI operation, we implemented a series of microbenchmarks measuring host access latency to target memory locations following an SDCI-initiated Memory Write transaction. The benchmark proceeds as follows:

1. Allocate a kernel-resident buffer and expose it to user space via the `mmap` interface provided by our custom kernel module.
2. Retrieve the steering tag for the target CPU using the kernel API described in Section 3.2.1.
3. Flush the targeted cache line(s) from the CPU caches so that subsequent CPU accesses must normally be serviced from main memory unless the device populates the cache.
4. Issue an SDCI MWr TLP that writes to the target physical address.
5. Wait briefly for the DMA transaction to complete, then measure the CPU access latency to the mapped address and verify data integrity.
6. Repeat the sequence for many iterations to compute average latency.

To simplify execution and eliminate process-level scheduling noise, the Go TLP library was embedded into a C-based benchmarking harness, so the entire measurement loop runs inside a single process without launching external helpers. We built the Go component using `-buildmode=c-shared` with Go 1.22.2 [Goo25] and compiled the final SDCI application with GCC 13.3.0 [GNU25] using the `-march=native` flag. An overview of the measurement implementation is provided in Listing 4.1.

```c
/*
 Measure the access latency (in CPU cycles) to ‘addr‘ after performing SDCI.
 - addr: pointer to the mapped user-space address that corresponds to the DMA target
 - fd:  file descriptor for the kernel device used to obtain steering tags, etc.
*/

uint64_t measure_sdci(void *addr, int fd) {
    uint64_t total_cycles = 0;

    for (int i = 0; i < ITERATIONS; i++) {
        // Flush the target cache line and serialize
        _mm_clflushopt(addr);
        _mm_mfence();
        _mm_lfence();

        // Obtain steering tag from kernel helper
        int tag = get_tag(fd);

        // Issue SDCI and obtain the expected value returned by the device
        int res = perform_sdci_and_return_value(tag);

        // Wait briefly to allow DMA to complete
        usleep(5000);

        // Measure CPU load time (cycles) to the address
        uint64_t cycles = measure_access_time((uint8_t *)addr);
        total_cycles += cycles;

        //verify correctness of written data
        uint32_t observed = *((volatile uint32_t *)addr);
        if ((uint32_t)res != observed) {
            printf("Data mismatch! Expected: 0x%08x, Got: 0x%08x\n", (uint32_t)res, observed);
        }
    }

    retrun total_cycles;
}
```

Listing 4.1: Memory access timing routine after SDCI.

Setup All experiments were performed on an AMD EPYC 9135 16 core processor [Adv25a], with the following cache hierarchy: L1d: 768 KiB, L2: 16 MiB and L3: 64 MiB. The benchmark running in user space was pinned to cores 2, 4, 6, 8 and 10 using `taskset` [Lov04].

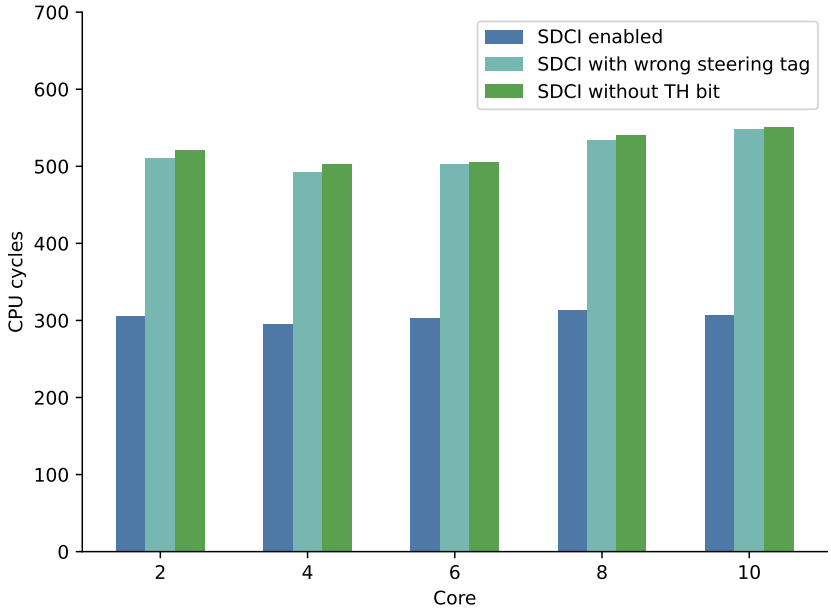


Figure 4.2: CPU access latency when using SDCI.

Inputs We evaluated three configurations. First, SDCI enabled with the steering tag matching the CPU on which the benchmark ran. Second, SDCI enabled but the steering tag pointed to a different CPU. Third, SDCI disabled with the `TH` bit unset. Each configuration was measured 100 times.

Results Figure 4.2 shows a clear separation between the correctly configured SDCI case and the other two. When SDCI was disabled or the steering tag was mismatched, access latency increased by roughly 200 cycles which is consistent with a main memory access [VSIH22]. With SDCI enabled and the correct steering tag, measured latency was approximately 300 cycles which aligns with an L3 cache access. We therefore conclude that SDCI was active and functioning as intended on our test system.

4.1.2 Overwriting Privileged Memory

Across all of our experiments, except in cases where the TLP payload length did not match the encoded length in the header, we observed no modifications to privileged memory. All attempted writes to the privileged memory region were completely ignored and had no effect on the RMP. We therefore strongly assume that the CS block prevents access to privileged memory through the use of a TMR register, as the writes appear to be silently discarded. These fences seem to be consistently

applied before the CPU is able to issue a `readblock` instruction to perfrom smart data cache injection.

In the experiment where the TLP payload length did not match the encoded length, the system crashed. This behavior is most likely due to a packet-type translation error when converting from TLP to an internal protocol used by the data fabric.

4.2 MMIO Access and Routing

Before applying any modifications to the MMIO-related registers, we used DMA to dump the first and a big part of the second MMIO regions in search of any interesting behavior. Most of the data returned was simply `0xFF`. For some requests, we received an *unsupported request* response. In a very small subset of memory regions, we observed data that appeared random but consistently reinitialized to the same value after each system reset.

This seemingly random data was also the only data we were able to overwrite. For all other regions, write attempts were silently discarded. Nevertheless, these overwrites had no observable side effects: the data changed as written, but we could not detect any impact on the running system.

4.2.1 IOHUB MMIO Register

Modifying the IOHUB MMIO register did not produce any observable impact. The dumped output for the MMIO regions remained identical after changing the IOHUB register, and the writability of the memory regions was unaffected. Only those regions that were writable before the modification remained writable afterwards. This suggests that the requests continued to be routed through the MMIO controller rather than being redirected to the memory controller.

Also no memory aliasing was observed. Requests targeting addresses beyond the physical limit returned `0xFF`. Furthermore, starting from address `0x400000000000`, requests were directly blocked by the IOMMU, which responded with a completion abort. This behavior is expected, as the upper address bits are reserved for encryption in AMD SEV-SNP. As a security feature, the IOMMU aborts such requests.

We conclude that the IOHUB register does not influence the routing of DMA requests.

4.2.2 IOMS Routing Register

Modifying the IOMS routing register led to noticeably different behavior. Nulling all ranges within the routing register caused all DMA requests to MMIO regions to return `0xFF`, including those that had previously returned other values. Additionally, no regions remained writable. All writes to the regions we had previously identified as writable were discarded.

Routing traffic directly to the CS block by modifying the `DstFabricID` consistently resulted in a system reset. We observed that when traffic is routed to the CS block through the IOMS routing register, the IOMS always issues a `SlaveErr`, which triggers a `SyncFlood` event and consequently forces a system reset. This occurs regardless of whether the targeted range corresponds to an MMIO region or to a valid, previously allocated memory region. Using an unused IOMS register for routing produced the same outcome.

Routing packets to the PIE block resulted in inconsistent behavior. In some cases, the returned data consisted of `0xFF` and `0x00`, while in others the system reset due to a `SyncFlood` event.

Lastly, when routing traffic to a CCM block, no `SlaveErr` was generated. Regardless of whether the requested range fell within MMIO space or allocated memory, the system consistently entered a `Completion-Wait` loop that timed out, leading to a system reset after approximately 2–3 seconds.

Performing the same experiments while modifying the IOHUB register did not influence the outcome.

We conclude that, beyond routing, additional mechanisms must be involved in handling DMA request packets, making it non-trivial to alter their routes.

5 Conclusion

In summary, we analyzed the security impact of Smart Data Cache Injection and MMIO rerouting on AMD SEV-SNP. To achieve this, we first enabled and verified SDCI functionality on our AMD EPYC 9135 platform using the Screamer PCIe Squirrel to generate raw DMA transactions. We then implemented several techniques to exercise SDCI under various conditions with the objective of overwriting the RMP. Finally, we modified MMIO routing registers in both the IOHUB and IOMS in an effort to redirect MMIO traffic toward the memory controller, aiming either to access MMIO regions through normal memory pathways or to create a memory alias.

Our findings show that all SDCI write attempts targeting privileged memory consistently failed, regardless of configuration or injection strategy. This suggests that a TMR validation step is always performed before any data is injected into the CPU cache hierarchy, effectively blocking unauthorized cache population or writeback into protected physical memory. Changes made to the IOHUB registers had no measurable effect on DMA routing or access permissions, indicating that these registers are likely limited to flagging MMIO transactions rather than enforcing access control. Modifying the IOMS routing registers produced stronger responses from the platform, ranging from completion timeouts and request aborts to full system resets, depending on the target destination. These reactions imply the presence of internal protection or fencing mechanisms that prevent the redirection of MMIO traffic into memory space. The underlying cause of these completion errors remains unclear due to the opaque nature of the hardware-level routing and protection logic, but the observed outcomes demonstrate that attempts to reroute or reinterpret MMIO transactions are effectively blocked.

Overall, we conclude that the tested AMD SEV-SNP platform provides strong and effective protection against the examined classes of attacks. The TMR enforcement and platform-level checks reliably prevent Smart Data Cache Injection from modifying privileged memory, while the tightly controlled and opaque handling of MMIO routing prevents redirection or aliasing of protected regions. Together, these mechanisms ensure that even low-level DMA-based manipulation remains ineffective against SEV-SNP's security boundaries.

List of Figures

2.1 Simplified architecture of a PCIe Root Complex and its connection hierarchy [Ota23]. 6
2.2 Structure of a PCIe TLP. 6
2.3 Structure of a PCIe Identifier (Bus, Device, and Function numbers). 7
2.4 Simplified DMA transaction flow within the data fabric, with and without Smart Data Cache Injection. 10
2.6 Example structure of a TLP containing TLP Processing Hints and steering information. 11
2.7 (a) RMP memory is writable before SNP initialization. (b) The PSP sets barriers blocking writes from x86 cores and devices. (c) After RMP initialization, x86 cores uses RMP semantics [SS25]. 13

3.2 Physical address layout example [CFHB22]. 22

4.2 CPU access latency when using SDCI. 27

List of Tables

2.5 Description of PCIe TLP Processing Hint Types 11

List of Listings

3.1 Example Go implementation for constructing and transmitting a Memory Write (MWr) TLP with an SDCI processing hint. 19
3.3 MMIO Address Control Register. 23

4.1 Memory access timing routine after SDCI. 26

Bibliography

[Adv15] Advanced Micro Devices, Inc.: *Amd i/o virtualization technology (iommu) specification.* Technical report, Advanced Micro Devices, Inc., February 2015. `http://support.amd.com/TechDocs/48882_IOMMU.pdf`, Specification.

[Adv20] Advanced Micro Devices, Inc.: *Amd sev-snp: Strengthening vm isolation with integrity protection and more.* Technical report, AMD, January 2020. `https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf`, White Paper.

[Adv23] Advanced Micro Devices (AMD): *Amd-aspfw.* `https://github.com/amd/AMD-ASPFW`, 2023. GitHub repository.

[Adv25a] Advanced Micro Devices, Inc.: *Amd epyc™ 9135.* `https://www.amd.com/de/products/processors/server/epyc/9005-series/amd-epyc-9135.html`, 2025.

[Adv25b] Advanced Micro Devices, Inc.: *Smart data cache injection (sdci) white paper.* Technical Report 58725, Rev. 1.00, AMD, February 2025. `https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/58725.pdf`, AMD Public Use.

[AMD23] AMD: *Amd opensil.* `https://github.com/openSIL/openSIL/tree/genoa_poc`, 2023. GitHub repository.

[ARM16] ARM Limited: *Arm system memory management unit architecture specification.* Technical report, ARM Limited, June 2016. `https://developer.arm.com/documentation`, Specification.

[CFHB22] Cfir Cohen, James Forshaw, Jann Horn, and Mark Brand: *Amd secure processor for confidential computing: Security review (gpz technical report).* Technical report, Google Project Zero & Google Cloud Security / AMD, May 2022. `https://storage.googleapis.com/gweb-uniblog-publish-prod/documents/AMD_GPZ-Technical_Report_FINAL_05_2022.pdf`, Technical report.

[DMWO+25] Jesse De Meulemeester, Luca Wilke, David Oswald, Thomas Eisenbarth, Ingrid Verbauwhede, and Jo Van Bulck: *Badram: Practical memory aliasing attacks on trusted execution environments*. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4117–4135, 2025.

[GNU25] GNU Project: *Gcc 13 release series*. `https://gcc.gnu.org/gcc-13/`, 2025.

[Goo23] Google: *go-pcie-tlp*. `https://github.com/google/go-pcie-tlp`, 2023. GitHub repository.

[Goo25] Google: *The go programming language*. `https://go.dev/`, 2025.

[HB17] Felicitas Hetzelt and Robert Buhren: *Security analysis of encrypted virtual machines*. In *Proceedings of the 13th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, VEE '17, page 129–142, New York, NY, USA, 2017. Association for Computing Machinery, ISBN 9781450349482. `https://doi.org/10.1145/3050748.3050763`.

[HDGT+17] Torsten Hoefler, Salvatore Di Girolamo, Konstantin Taranov, Ryan E Grant, and Ron Brightwell: *spin: High-performance streaming processing in the network*. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16, 2017.

[Int14] Intel Corporation: *Intel virtualization technology for directed i/o architecture specification*. Technical report, Intel Corporation, 2014. `https://www.intel.com/content/www/us/en/virtualization/virtualization-technology-for-directed-io-spec.html`, Specification.

[Kap17] David Kaplan: *Protecting vm register state with sev-es*. Technical report, Advanced Micro Devices, Inc., February 2017. `https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/Protecting-VM-Register-State-with-SEV-ES.pdf`, White Paper.

[kc24] The Linux kernel community: *Pat (page attribute table) — linux kernel documentation*. `https://www.kernel.org/doc/Documentation/x86/pat.txt`, 2024.

[Lam25] LambdaConcept: *Screamer pcie squirrel*. `https://shop.lambdaconcept.com/home/50-screamer-pcie-squirrel.html`, 2025.

[Lar25] Michael Larabel: *Linux 6.16 will begin reporting the cause of your amd zen system being reset/rebooted*. https://www.phoronix.com/news/AMD-Report-Previous-Reset-Cause, May 2025.

[Lin23] Linus Torvalds and Linux community: *Linux kernel source (v6.15-rc5)*. https://github.com/torvalds/linux/tree/v6.15-rc5, 2023. GitHub repository.

[Lov04] Robert M. Love: *taskset(1) — linux manual page*. https://man7.org/linux/man-pages/man1/taskset.1.html, 2004. Part of the util-linux package.

[MHHW18] Mathias Morbitzer, Manuel Huber, Julian Horsch, and Sascha Wessel: *Severed: Subverting amd's virtual machine encryption*. In *Proceedings of the 11th European Workshop on Systems Security*, EuroSys '18, page 1–6. ACM, April 2018. http://dx.doi.org/10.1145/3193111.3193112.

[MPR+21] Mathias Morbitzer, Sergej Proskurin, Martin Radev, Marko Dorfhuber, and Erick Quintanar Salas: *Severity: Code injection attacks against encrypted virtual machines*, 2021. https://arxiv.org/abs/2105.13824.

[MRG+19] A. Markettos, Colin Rothwell, Brett Gutstein, Allison Pearce, Peter Neumann, Simon Moore, and Robert Watson: *Thunderclap: Exploring vulnerabilities in operating system iommu protection via dma from untrustworthy peripherals*. January 2019.

[Ota23] John Otander: *A practical tutorial on pcie for total beginners on windows (part 1)*. https://ctf.re/windows/kernel/pcie/tutorial/2023/02/14/pcie-part-1/, Feb 2023.

[PDH25] Petar Paradžik, Ante Derek, and Marko Horvat: *Formal security analysis of the amd sev-snp software interface*, 2025. https://arxiv.org/abs/2403.10296.

[PS19] PCI-SIG: *Pci specification*. Technical report, Eetop / Picture.ICZ Hiku, 2019. https://picture.iczhiku.com/resource/eetop/SYkDTqhOLhpUTnMx.pdf.

[Rot17] Colin Rothwell: *Exploitation from Malicious PCI Express Peripherals*. PhD thesis, September 2017.

[SS25] Benedict Schlüter and Shweta Shinde: *Rmpocalypse: How a catch-22 breaks amd sev-snp*. In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '25. Association for Computing Machinery, 2025.

[SSBS24] Benedict Schlüter, Supraja Sridhara, Andrin Bertschi, and Shweta Shinde: *Wesee: Using malicious #vc interrupts to break amd sev-snp.* In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4220–4238, 2024.

[SWS25] Benedict Schlüter, Christoph Wech, and Shweta Shinde: *Heracles: Chosen plaintext attack on amd sev-snp.* In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '25. Association for Computing Machinery, 2025.

[VSIH22] Markus Velten, Robert Schöne, Thomas Ilsche, and Daniel Hackenberg: *Memory performance of amd epyc rome and intel cascade lake sp server processors.* In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering*, ICPE '22, page 165–175. ACM, April 2022. `http://dx.doi.org/10.1145/3489525.3511689`.

[vTHtLkc24] Eric van Tassell, Wei Huang, and the Linux kernel community: *Tph support — the linux kernel documentation.* `https://docs.kernel.org/PCI/tph.html`, 2024.

[YLStLkc16] Fenghua Yu, Tony Luck, Vikas Shivappa, and the Linux kernel community: *User interface for resource control (resctrl) — linux kernel documentation.* `https://www.kernel.org/doc/Documentation/x86/resctrl.rst`, 2016.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz[1] verwendet.

☒ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

Evaluating the Security of AMD SEV-SNP Against Cache-Directed DMA and MMIO Rerouting Attacks

Verfasst von:
Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):	**Vorname(n):**
Vezzù	Alessandro

Ich bestätige mit meiner Unterschrift:
- Ich habe mich an die Regeln des «Zitierleitfadens» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum	**Unterschrift(en)**
Pfungen, 18.11.2025	Vezzù

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

[1] Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html und https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html (Änderungen vorbehalten).